Transformación del modelo EER al modelo relacional — Reglas 1 a 9

Regla 1. Toda entidad fuerte se transforma en una tabla

Se toma cada entidad fuerte, se crea una tabla con sus atributos simples y se declara como clave primaria el identificador de la entidad. En el modelo hay cinco entidades fuertes: `PRODUCTO`, `CATEGORIA`, `FABRICANTE`, `COMPRA` y `VENTA`.

1.1 Tabla `PRODUCTO`

Columna	Tipo propuesto	Restricciones
`id`	`INT`	`PRIMARY KEY`, `NOT NULL`
`nombre`	`VARCHAR(120)`	Recomendado: `NOT NULL`
`presentacion`	`VARCHAR(80)`	Recomendado: `NOT NULL`
`contenido`	`VARCHAR(100)`	Puede ser `NULL` hasta confirmar obligatoriedad
`precioVenta`	`DECIMAL(10,2)`	Recomendado: `NOT NULL`, `CHECK (precioVenta >= 0)`
`estado`	`VARCHAR(20)`	Recomendado: `NOT NULL`
`categoria_id`	`INT`	`FOREIGN KEY` a `CATEGORIA(id)`; recomendado `NOT NULL`
`fabricante_id`	`INT`	`FOREIGN KEY` a `FABRICANTE(id)`; recomendado `NOT NULL`

Resultado: tabla `PRODUCTO`, con `id` como PK y `categoria_id` y `fabricante_id` como FKs.

1.2 Tabla `CATEGORIA`

Columna	Tipo propuesto	Restricciones
`id`	`INT`	`PRIMARY KEY`, `NOT NULL`
`nombre`	`VARCHAR(100)`	Recomendado: `NOT NULL`
`descripcion`	`VARCHAR(255)`	Puede ser `NULL`
`estado`	`VARCHAR(20)`	Recomendado: `NOT NULL`

Resultado: tabla `CATEGORIA`, con `id` como PK.

1.3 Tabla `FABRICANTE`

Columna	Tipo propuesto	Restricciones
`id`	`INT`	`PRIMARY KEY`, `NOT NULL`
`nombre`	`VARCHAR(120)`	Recomendado: `NOT NULL`
`correo`	`VARCHAR(254)`	`UNIQUE` según la marca `UK` del modelo; `NOT NULL` solo si el negocio lo exige
`sitioWeb`	`VARCHAR(255)`	Puede ser `NULL`
`pais`	`VARCHAR(80)`	Puede ser `NULL` hasta confirmar obligatoriedad

Columna	Tipo propuesto	Restricciones
estado	VARCHAR(80)	Puede ser NULL hasta confirmar obligatoriedad
codigoPostal	VARCHAR(20)	Puede ser NULL hasta confirmar obligatoriedad

Resultado: tabla FABRICANTE, con id como PK y correo como UNIQUE según la indicación UK del diagrama. No se incluye telefono porque se convierte en una tabla aparte al aplicar la Regla 3.

1.4 Tabla COMPRA

Columna	Tipo propuesto	Restricciones
id	INT	PRIMARY KEY, NOT NULL
fecha	DATE	Recomendado: NOT NULL
subtotal	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (subtotal >= 0)
descuento	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (descuento >= 0)
impuesto	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (impuesto >= 0)
total	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (total >= 0)
estado	VARCHAR(20)	Recomendado: NOT NULL

Resultado: tabla COMPRA, con id como PK.

1.5 Tabla VENTA

Columna	Tipo propuesto	Restricciones
id	INT	PRIMARY KEY, NOT NULL
fechaHora	TIMESTAMP	Recomendado: NOT NULL
subtotal	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (subtotal >= 0)
descuento	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (descuento >= 0)
impuesto	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (impuesto >= 0)
total	DECIMAL(10,2)	Recomendado: NOT NULL, CHECK (total >= 0)
metodoPago	VARCHAR(40)	Recomendado: NOT NULL
estado	VARCHAR(20)	Recomendado: NOT NULL

Resultado: tabla VENTA, con id como PK.

Regla 2. Los atributos compuestos se descomponen en componentes simples

En el EER, DIRECCIÓN es un atributo compuesto de FABRICANTE, con los componentes país, estado y códigoPostal.

Transformación aplicada

- Se crean las columnas `pais`, `estado` y `codigoPostal` en `FABRICANTE`.
- No se crea una columna llamada `DIRECCIÓN`.
- No se crea una tabla independiente para la dirección, ya que la regla indica descomponerla en columnas simples.
- Se conservan como tipos propuestos `VARCHAR(80)`, `VARCHAR(80)` y `VARCHAR(20)`, respectivamente; el modelo original no fija tipos.

Resultado de la Regla 2:

`FABRICANTE(id, nombre, correo, sitioWeb, pais, estado, codigoPostal)`

> Nota: `estado` aquí es el componente geográfico de DIRECCIÓN. No debe confundirse con los atributos `estado` de PRODUCTO, CATEGORIA, LOTE, COMPRA y VENTA.

Regla 3. Los atributos multivaluados se convierten en una tabla aparte

Tabla nueva: `FABRICANTE_TELEFONO`

Columna	Tipo propuesto	Restricciones
`fabricante_id`	`INT`	`NOT NULL`, FK a `FABRICANTE(id)` y parte de la PK
`telefono`	`VARCHAR(25)`	`NOT NULL`, parte de la PK

Clave primaria: `PRIMARY KEY (fabricante_id, telefono)`.

Resultado de la Regla 3:

`FABRICANTE_TELEFONO(fabricante_id, telefono)`

Esta clave compuesta permite registrar varios teléfonos para un mismo fabricante y evita repetir el mismo teléfono para el mismo fabricante. No se declara `telefono` como único globalmente porque el EER no establece esa restricción.

Reglas 4 a 7. Transformación de las relaciones

Regla 4. Relaciones uno a muchos (1:N)

En una relación 1:N, la clave primaria de la entidad del lado uno se incorpora como clave foránea a la tabla del lado muchos. La FK es `NOT NULL` cuando cada registro del lado muchos debe participar obligatoriamente; acepta `NULL` cuando la participación mínima en esa relación es cero.

4.1 `CATEGORIA` — clasifica — `PRODUCTO`

- **Cardinalidades según el EER:** una categoría clasifica de 1 a N productos; cada producto pertenece a exactamente una categoría (`(1,N)` en CATEGORÍA y `(1,1)` en PRODUCTO).
- **Regla aplicada:** Regla 4, 1:N.
- **FK propagada:** `CATEGORIA.id` → `PRODUCTO.categoria_id`.
- **Restricción:** `PRODUCTO.categoria_id INT NOT NULL`, además de `FOREIGN KEY`.
- **Tablas resultantes:** se mantienen `CATEGORIA` y `PRODUCTO`; no se crea una tabla adicional para `clasifica`.
- **Representación resumida:** `PRODUCTO(..., categoria_id FK NOT NULL)`.

4.2 FABRICANTE — fabrica — PRODUCTO

- **Cardinalidades según el EER:** un fabricante puede fabricar de 0 a N productos; cada producto se asocia con exactamente un fabricante (`(0,N)` en FABRICANTE y `(1,1)` en PRODUCTO).
- **Regla aplicada:** Regla 4, 1:N.
- **FK propagada:** `FABRICANTE.id` → `PRODUCTO.fabricante_id`.
- **Restricción:** `PRODUCTO.fabricante_id INT NOT NULL`, además de `FOREIGN KEY`. El mínimo cero del lado FABRICANTE significa que un fabricante puede no tener productos asociados; no vuelve opcional la FK del producto.
- **Tablas resultantes:** `FABRICANTE` y `PRODUCTO`; no se crea una tabla para `fabrica`.
- **Representación resumida:** `PRODUCTO(..., fabricante_id FK NOT NULL)`.

4.3 PRODUCTO — posee — LOTE (relación identificadora)

- **Cardinalidades interpretadas del EER:** un producto puede tener de 0 a N lotes; cada lote pertenece a exactamente un producto.
- **Regla aplicada:** Regla 4, 1:N, con la particularidad de que `LOTE` es una entidad débil y `posee` es su relación identificadora.
- **FK propagada:** `PRODUCTO.id` → `LOTE.producto_id`.
- **Restricciones:** `LOTE.producto_id NOT NULL`, FK a `PRODUCTO(id)` y además parte de la clave primaria compuesta de LOTE.
- **PK de LOTE:** `(producto_id, numeroLote)`, donde `numeroLote` es la clave parcial del EER.
- **Tablas resultantes:** `PRODUCTO` y `LOTE`; no se agrega una tercera tabla para `posee`.
- **Representación resumida:** `LOTE(producto_id PK/FK, numeroLote PK, ...)`.

4.4 COMPRA — proviene de — LOTE

- **Regla aplicada:** Regla 4, 1:N, interpretando que una compra puede estar asociada con varios lotes y que la referencia a una compra en un lote es opcional.
- **FK propagada:** `COMPRA.id` → `LOTE.compra_id`.
- **Restricción:** `LOTE.compra_id INT NULL`, FK a `COMPRA(id)`. Se permite `NULL` por la marca `*` del modelo relacional; el diagrama no proporciona una leyenda del asterisco, así que esta nulabilidad debe confirmarse con el profesor o el criterio del proyecto.
- **Tablas resultantes:** `COMPRA` y `LOTE`; no se crea una tabla para la relación si sigue siendo binaria 1:N.
- **Representación resumida:** `LOTE(..., compra_id FK NULL)`.

4.5 COMPRA — contiene — DETALLE_COMPRA

- **Cardinalidad interpretada:** una compra puede tener varios detalles; cada detalle de compra pertenece a una sola compra.
- **Regla aplicada:** Regla 4, 1:N.
- **FK propagada:** `COMPRA.id` → `DETALLE_COMPRA.compra_id`.
- **Restricciones:** `DETALLE_COMPRA.compra_id NOT NULL`, FK y parte de la PK compuesta.
- **Tablas resultantes:** `COMPRA` y `DETALLE_COMPRA`; el detalle ya existe en el modelo relacional.
- **Representación resumida:** `DETALLE_COMPRA(compra_id PK/FK, producto_id PK/FK, ...)`.

4.6 PRODUCTO — corresponde a — DETALLE_COMPRA

- **Cardinalidad interpretada:** un producto puede aparecer en muchos detalles de compra; cada detalle se refiere a un solo producto.
- **Regla aplicada:** Regla 4, 1:N.
- **FK propagada:** `PRODUCTO.id` → `DETALLE_COMPRA.producto_id`.
- **Restricciones:** `DETALLE_COMPRA.producto_id NOT NULL`, FK y parte de la PK compuesta.
- **Tablas resultantes:** `PRODUCTO` y `DETALLE_COMPRA`; no se añade una tabla extra para `corresponde`.

4.7 VENTA — contiene — DETALLE_VENTA

- **Cardinalidad interpretada:** una venta puede tener varios detalles; cada detalle de venta pertenece a una sola venta.
- **Regla aplicada:** Regla 4, 1:N.
- **FK propagada:** VENTA.id → DETALLE_VENTA.venta_id.
- **Restricciones:** DETALLE_VENTA.venta_id NOT NULL, FK y parte de la PK compuesta.
- **Tablas resultantes:** VENTA y DETALLE_VENTA; no se crea una tabla adicional para contiene.

4.8 PRODUCTO — corresponde a — DETALLE_VENTA

- **Cardinalidad interpretada:** un producto puede aparecer en muchos detalles de venta; cada detalle se refiere a un solo producto.
- **Regla aplicada:** Regla 4, 1:N.
- **FK propagada:** PRODUCTO.id → DETALLE_VENTA.producto_id.
- **Restricciones:** DETALLE_VENTA.producto_id NOT NULL, FK y parte de la PK compuesta.
- **Tablas resultantes:** PRODUCTO y DETALLE_VENTA; no se crea una tabla extra para corresponde.

Regla 5. Relaciones muchos a muchos (M:N)

Una relación M:N requiere una tabla asociativa con las claves de las entidades participantes como FKs y, habitualmente, como PK compuesta. Los atributos propios de la relación también se guardan en esa tabla.

5.1 Asociación COMPRA PRODUCTO, resuelta con DETALLE_COMPRA

- **Relación conceptual:** una compra puede incluir varios productos y un producto puede aparecer en muchas compras; es una asociación M:N.
- **Regla aplicada:** Regla 5. La asociación se materializa en DETALLE_COMPRA.
- **Claves propagadas:** COMPRA.id se convierte en DETALLE_COMPRA.compra_id; PRODUCTO.id se convierte en DETALLE_COMPRA.producto_id.
- **Atributos propios de la asociación:** cantidad, precioUnitario, descuento.
- **Clave primaria resultante:** (compra_id, producto_id).
- **Restricciones:** ambas columnas son NOT NULL y FKs a COMPRA(id) y PRODUCTO(id).
- **Tabla resultante:** DETALLE_COMPRA(compra_id, producto_id, cantidad, precioUnitario, descuento).

5.2 Asociación VENTA PRODUCTO, resuelta con DETALLE_VENTA

- **Relación conceptual:** una venta puede incluir varios productos y un producto puede aparecer en muchas ventas; es una asociación M:N.
- **Regla aplicada:** Regla 5. La asociación se materializa en DETALLE_VENTA.
- **Claves propagadas:** VENTA.id → DETALLE_VENTA.venta_id; PRODUCTO.id → DETALLE_VENTA.producto_id.
- **Atributos propios de la asociación:** cantidad, precioUnitario, descuento.
- **Clave primaria resultante:** (venta_id, producto_id).
- **Restricciones:** ambas columnas son NOT NULL y FKs a VENTA(id) y PRODUCTO(id).
- **Tabla resultante:** DETALLE_VENTA(venta_id, producto_id, cantidad, precioUnitario, descuento).

Regla 6. Relaciones uno a uno (1:1)

6.1 PRODUCTO — MEDICAMENTO

- **Estrategia elegida:** conservar una tabla para el supertipo y una para el subtipo, compartiendo la clave. Es preferible a fusionarlas porque MEDICAMENTO tiene atributos propios y la jerarquía del EER es disjunta y total.
- **Propagación:** PRODUCTO.id → MEDICAMENTO.producto_id.

- **Restricciones:** `MEDICAMENTO.producto_id` es al mismo tiempo PK y FK, por lo que es `NOT NULL` y único.
- **Tabla resultante:** `MEDICAMENTO(producto_id PK/FK, principioActivo, concentracion, requiereReceta)`.
- **Tabla independiente de relación:** no se crea.

6.2 `PRODUCTO` — `OTRO_PRODUCTO`

- **Estrategia elegida:** la misma estrategia de tabla por subtipo con clave compartida.
- **Propagación:** `PRODUCTO.id` → `OTRO_PRODUCTO.producto_id`.
- **Restricciones:** `OTRO_PRODUCTO.producto_id` es PK y FK, por lo tanto `NOT NULL` y único.
- **Tabla resultante:** `OTRO_PRODUCTO(producto_id PK/FK, tipoProducto, unidadVenta)`.
- **Tabla independiente de relación:** no se crea.

Regla 7. Relaciones ternarias o de orden superior

- **Resultado del análisis:** no se identifica una relación ternaria

---

Regla 8. Entidades débiles

La entidad débil no se identifica únicamente con una clave propia: necesita la clave de la entidad o entidades propietarias. En el EER se reconoce por el doble rectángulo y, cuando aparece, su relación identificadora por el doble rombo. El modelo relacional ya contiene tablas para las tres entidades dibujadas con doble rectángulo: `LOTE`, `DETALLE_COMPRA` y `DETALLE_VENTA`; por tanto, no es necesario crear tablas duplicadas.

8.1 `LOTE`, identificado por `PRODUCTO`

- **Dependencia:** identificadora mediante `PRODUCTO` — `posee` — `LOTE`.
- **Discriminador / clave parcial:** `numeroLote`, señalado expresamente como clave parcial en el EER.
- **Clave del propietario:** `PRODUCTO.id`, propagada como `LOTE.producto_id`.
- **Clave primaria resultante:** `PRIMARY KEY (producto_id, numeroLote)`.
- **Restricciones:** ambos componentes son `NOT NULL`; `producto_id` es además FK a `PRODUCTO(id)`. Se recomienda `ON DELETE CASCADE` en esta FK porque un lote no puede seguir identificado si desaparece el producto propietario.
- **Otra FK:** `compra_id` aparece en el modelo relacional con un asterisco, interpretado aquí como opcional. No forma parte de la PK ni de la identificación de `LOTE`; por ello, no debe confundirse con la FK identificadora `producto_id`. Si se decide que una compra puede eliminarse sin eliminar los lotes, `ON DELETE SET NULL` podría ser apropiado, pero debe validarse con la regla de negocio.
- **Tabla resultante:** `LOTE(producto_id PK/FK, numeroLote PK, compra_id FK NULL, fechaFabricacion, fechaCaducidad, cantidadRecibida, disponibles, estado)`.

8.2 `DETALLE_COMPRA`, dependiente de `COMPRA` y asociado a `PRODUCTO`

- **Identificación en el modelo relacional:** la clave queda formada por la combinación de `compra_id` y `producto_id`. Ambos campos son `NOT NULL`, son FKs y forman la PK compuesta.
- **Discriminador:** el EER no muestra un atributo parcial específico para `DETALLE_COMPRA`. En la tabla relacional recibida, `producto_id` completa la clave compuesta junto con `compra_id`; no se debe inventar otro discriminador sin cambiar el modelo.
- **Atributos propios:** `cantidad`, `precioUnitario`, `descuento`.
- **Restricciones recomendadas:** `FOREIGN KEY (compra_id) REFERENCES COMPRA(id) ON DELETE CASCADE` y `FOREIGN KEY (producto_id) REFERENCES PRODUCTO(id) ON DELETE CASCADE`, porque un detalle no tiene sentido sin su compra ni el producto que registra.
- **Tabla resultante:** `DETALLE_COMPRA(compra_id PK/FK, producto_id PK/FK, cantidad, precioUnitario, descuento)`, con `PRIMARY KEY (compra_id, producto_id)`.

- **¿Tabla nueva?:** no; ya existe y cumple la función de asociación entre compras y productos. La PK de ambas FKs debe mantenerse.

8.3 `DETALLE_VENTA`, dependiente de `VENTA` y asociado a `PRODUCTO`

- **Identificación en el modelo relacional:** `PRIMARY KEY (venta_id, producto_id)`, formada por las dos FKs.
- **Discriminador:** el EER no presenta una clave parcial propia. En el modelo relacional actual, `producto_id` junto con `venta_id` compone la clave; no se agrega una columna artificial para el discriminador.
- **Atributos propios:** `cantidad`, `precioUnitario`, `descuento`.
- **Restricciones recomendadas:** ambas FKs son `NOT NULL` y con `ON DELETE CASCADE`, ya que el detalle depende de la venta y del producto registrado.
- **Tabla resultante:** `DETALLE_VENTA(venta_id PK/FK, producto_id PK/FK, cantidad, precioUnitario, descuento)`, con `PRIMARY KEY (venta_id, producto_id)`.

Regla 9. Jerarquías de generalización y especialización

9.1 Jerarquía identificada

El EER presenta una jerarquía ISA con `PRODUCTO` como supertipo y `MEDICAMENTO` y `OTRO_PRODUCTO` como subtipos. La leyenda del propio diagrama indica **disjunta (d) y total (t)**.

- **Disyunción:** disjunta. Un producto no puede pertenecer simultáneamente a `MEDICAMENTO` y `OTRO_PRODUCTO`.
- **Completitud:** total. Todo `PRODUCTO` debe ser de uno de esos dos subtipos.

9.2 Estrategia elegida: A, tabla por supertipo y por subtipo

Se elige la estrategia A, que también es la representación que ya utiliza `Modelo_Relacional.drawio`.

Justificación: `PRODUCTO` tiene los atributos generales compartidos, mientras que `MEDICAMENTO` cuenta con `principioActivo`, `concentracion` y `requiereReceta`, y `OTRO_PRODUCTO` con `tipoProducto` y `unidadVenta`. Separar las tablas evita columnas específicas del subtipo con muchos valores `NULL` y mantiene los atributos propios agrupados por tipo. Para búsquedas generales (catálogo, nombre, presentación, precio o estado) se consulta `PRODUCTO`; cuando se necesitan datos particulares de medicamentos u otros productos, se realiza un `JOIN` con el subtipo correspondiente. Ese costo de combinación es razonable para este diseño.

No se selecciona la estrategia B porque una sola tabla exigiría un discriminador y columnas específicas de ambos subtipos que quedarían nulas según el tipo. Tampoco se selecciona la estrategia C porque repetiría los atributos comunes de `PRODUCTO` en ambos subtipos, además de duplicar la información general.

9.3 Tablas resultantes, claves y restricciones

`PRODUCTO` — supertipo

```
PRODUCTO(
  id PK,
  nombre,
  presentacion,
  contenido,
  precioVenta,
  estado,
  categoria_id FK -> CATEGORIA.id,
  fabricante_id FK -> FABRICANTE.id
)
```

Todos los atributos comunes permanecen en el supertipo. `id` es la PK. `categoria_id` y `fabricante_id` permanecen como FKs de las relaciones 1:N ya documentadas. **No se agrega un discriminador** porque la estrategia A mantiene los subtipos en sus propias tablas.

`MEDICAMENTO` — **subtipo**

```
MEDICAMENTO(
  producto_id PK/FK -> PRODUCTO.id,
  principioActivo,
  concentracion,
  requiereReceta
)
```

`producto_id` es a la vez PK y FK, por lo que es `NOT NULL` y único. Se recomienda `ON DELETE CASCADE` desde `PRODUCTO`, ya que una fila de subtipo no puede subsistir sin la fila del supertipo.

`OTRO_PRODUCTO` — **subtipo**

```
OTRO_PRODUCTO(
  producto_id PK/FK -> PRODUCTO.id,
  tipoProducto,
  unidadVenta
)
```

`producto_id` es a la vez PK y FK, por lo que es `NOT NULL` y único. Se recomienda `ON DELETE CASCADE` desde `PRODUCTO` por la misma dependencia.

9.4 Restricciones para que la jerarquía sea realmente disjunta y total

La PK/FK compartida asegura que cada fila de subtipo corresponda a un producto existente y que un producto aparezca como máximo una vez en cada tabla de subtipo. **Por sí sola no garantiza** las dos reglas globales de la jerarquía:

- **Disyunción:** debe impedirse que un mismo `producto_id` aparezca tanto en `MEDICAMENTO` como en `OTRO_PRODUCTO`. Normalmente se implementa con triggers o lógica transaccional de la aplicación, salvo que el gestor disponga de otra forma de imponer esa restricción entre tablas.
- **Totalidad:** cada fila de `PRODUCTO` debe tener una fila en exactamente una de las dos tablas de subtipo. La aplicación debe crear el producto y su subtipo en una misma transacción, y se debe validar que ningún producto quede sin subtipo. Si se quiere garantizarlo desde la base de datos, hace falta una restricción diferida/trigger apropiado para el gestor utilizado.